



クラウドの不要なリソースと削減可能額を可視化する ダッシュボードを作って全社公開した話

NTT Tech Conference 2026 ☁ 2026/02/18

ドコモ・テクノロジー サービスインテグレーション事業部 スマートインテグレーション部
奥田 絢香



自己紹介

奥田 絢香 (Ayaka Okuda)

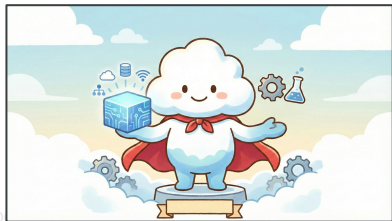
担当業務: CCoE (Cloud Center of Excellence)

→ 新卒入社以降、ずっとCCoEをしています！

- パブリッククラウド関連
 - クラウドコスト最適化 
 - ドコモ・クラウドパッケージ ...etc
- 共通基盤
 - Google Workspace
 - New Relic ...etc
- docomo R&D OPEN LAB ODAIBA

最近ハマっていること

- Geminiで作るインフォグラフィック



Nano Banana Pro  に
作ってもらった
ドコモ CCoEの
イメージキャラクター 

**docomo
CLOUD CENTER OF
EXCELLENCE**

共通基盤



**docomo R&D
OPEN LAB ODAIBA**





クラウドコスト、チリツモ浪費を防げ！ ドコモ CCoEの可視化ダッシュボード



パブリッククラウドの罠：
従量課金とチリツモ浪費



ドコモ CCoEの挑戦：
2,400+ AWSアカウントの塵を可視化！

ドコモグループ全2,400+アカウントを管理



ドコモグループ全2,400+アカウントを管理

全アカウントの不要リソース&
削減可能額を可視化する
ダッシュボードを構築・公開！

生成AIを活用！
1人で1ヶ月で構築完了！



このセッションで話すこと



『クラウドの不要な
リソース』とは？



『可視化ダッシュボード』
はどんなもの？



ダッシュボードは
どうやって構築した？



学べること



クラウド利用に
おけるコスト
削減のポイント



ドコモ CCoEの
クラウドコスト
最適化の取り組み

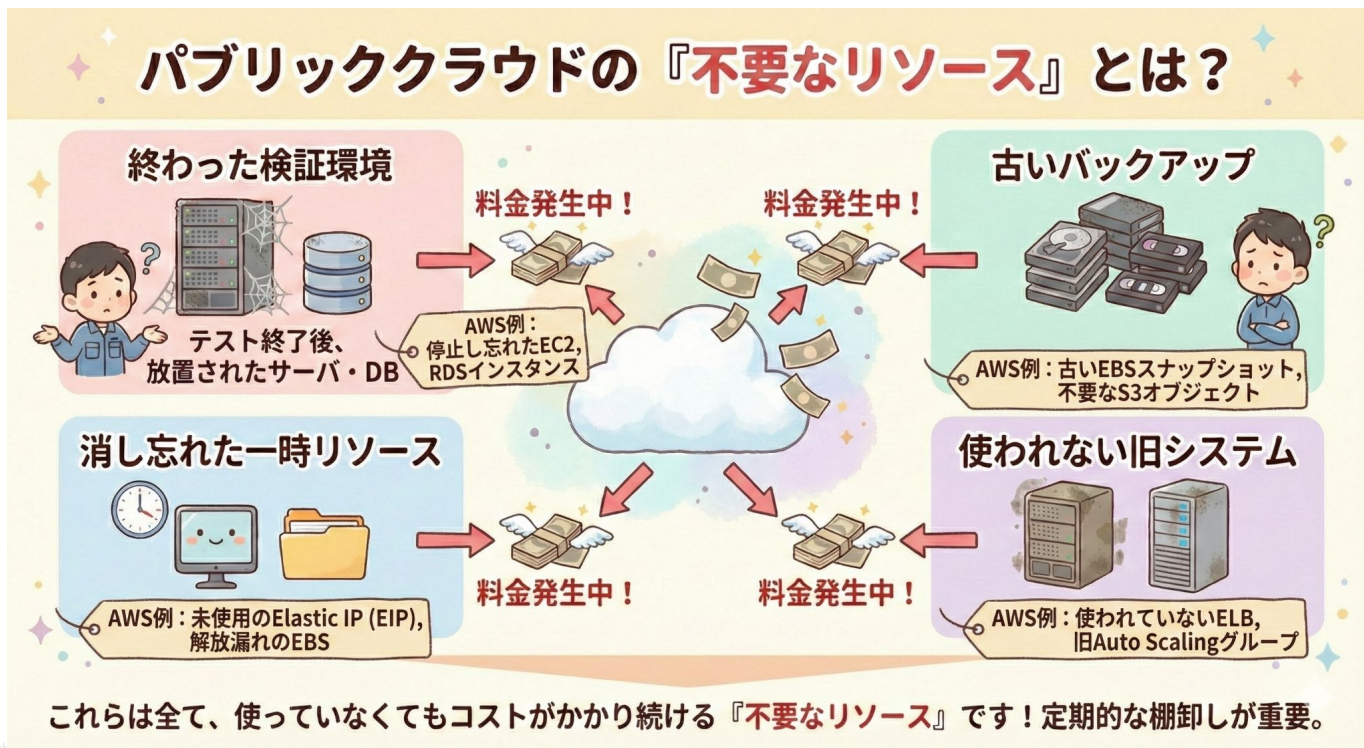


「クラウドの不要なリソース」とは？



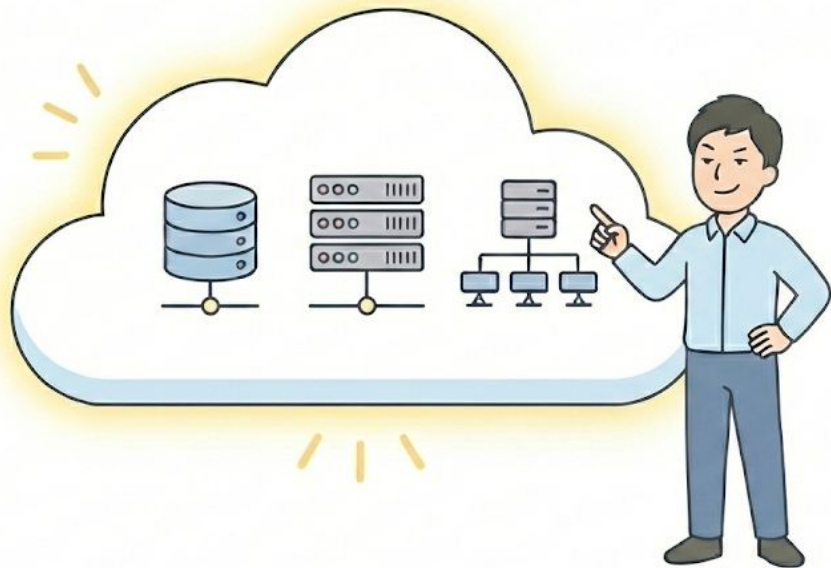
「クラウドの不要なリソース」とは？

- ✓ 本来もう必要ないのにアカウント内で存在したままになっているリソース
- パブリッククラウドは基本的に従量課金制なので、使ったら使った分だけ利用料金が発生する



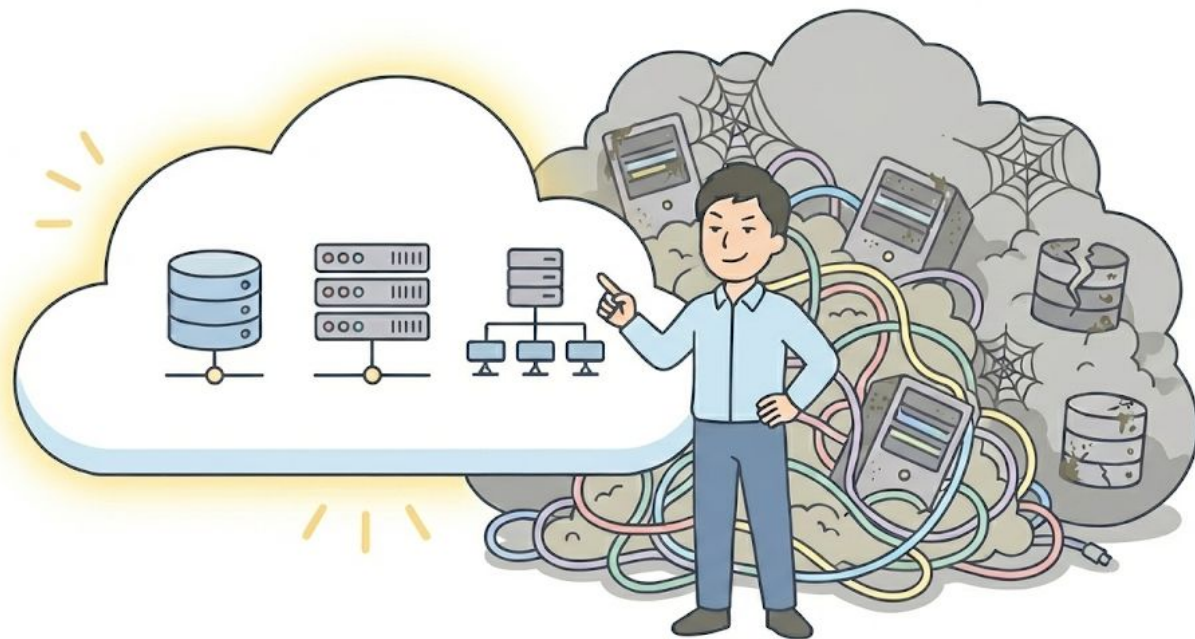
みなさんに質問です。

- ✓ 使用しているAWSアカウントで「不要なリソース」は 1つもない！と言えますか？



みなさんに質問です。

- ✓ 気づかないうちにあなたの AWS アカウントにも不要なリソースが溜まっているかもしれません。

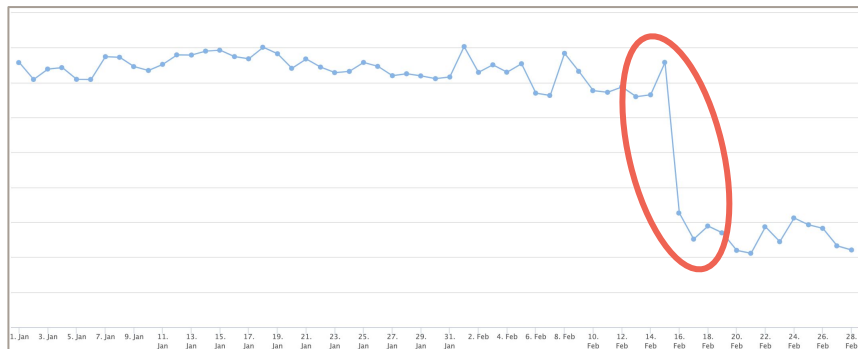


ドコモ(CCoE)での不要なリソースによる過去事例

- ✓ チームで使っている・長期間使い続けていると気付かぬうちに不要なリソースが溜まっていることが ...
- ✓ 継続的に不要なリソースを自分たちで確認し、対応し続けることは意外と難しい

不要なリソースによる過去事例①

- 過去のストレージやバックアップを放置
 - 複数名で使用するアカウントであったため、誰のリソースか不明なものが多々存在
- リソースの総点検を実施したところ、発生していた利用料金のうち約60%分が不要なリソースによるものだった



不要なリソースによる過去事例②

- 検証用や旧環境のリソース(サーバやDBなど)を起動した状態で放置
 - 開発速度を優先した結果、リソース整理が蔑ろにされていた
- 現環境では使用していないリソースを削除することでコスト削減

EC2(サーバ)	古い踏み台用サーバ
Aurora(DB)	構成変更前のデータベース
ElastiCache(Redis)	検証用リソース
EBS(ストレージ)	古いストレージのバックアップ



「可視化ダッシュボード」とは？

「不要なリソースと削減可能額の可視化」の取り組み

- ✓ パブリッククラウドの不要なリソースと削減可能額を一覧表示する「Cloud Frugal Dashboard」を構築
- ✓ 2025/8/18の社内リリース以降、約20,000個のリソース削減を達成
- ✓ ドコモグループ全体に対して公開し、いつでもユーザ自身で検出内容を確認可能（全検出内容を誰でも確認可能）

Cloud Frugal Dashboardが現状対応している6つの項目

- EC2: UnusedBox
 - EC2オンデマンドキャパシティ予約をしているが未使用
- EC2: InstanceIdleOrUnused
 - 起動したままで利用していない EC2
- EBS: CreateSnapshotAndDelete
 - EC2インスタンスにアタッチされていない EBS
- EBS: DetachAndCreateSnapshotAndDelete
 - EC2インスタンスにアタッチされているがインスタンスが利用されていない EBS
- RDS: ExtendedSupport
 - RDS(Aurora含)の延長サポートを利用している
- RDS: InstanceIdleOrUnused
 - 起動したままで利用していない RDS



Amazon Elastic Compute Cloud
(Amazon EC2)



Amazon Elastic Block Store
(Amazon EBS)



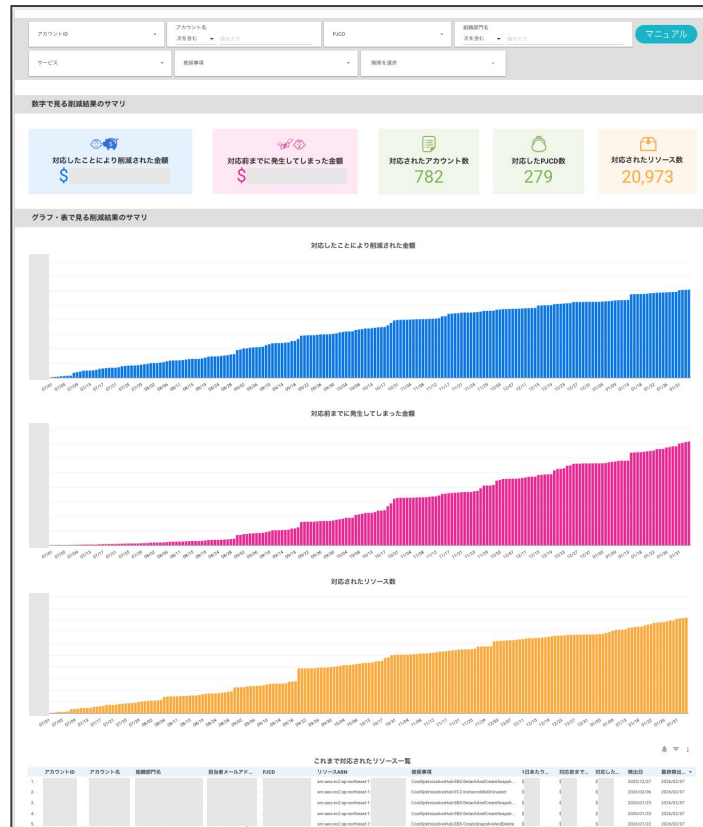
Amazon Relational Database Service
(Amazon RDS)

Cloud Frugal Dashboard 画面イメージ

推定削減可能額と対応すべきリソースのページ

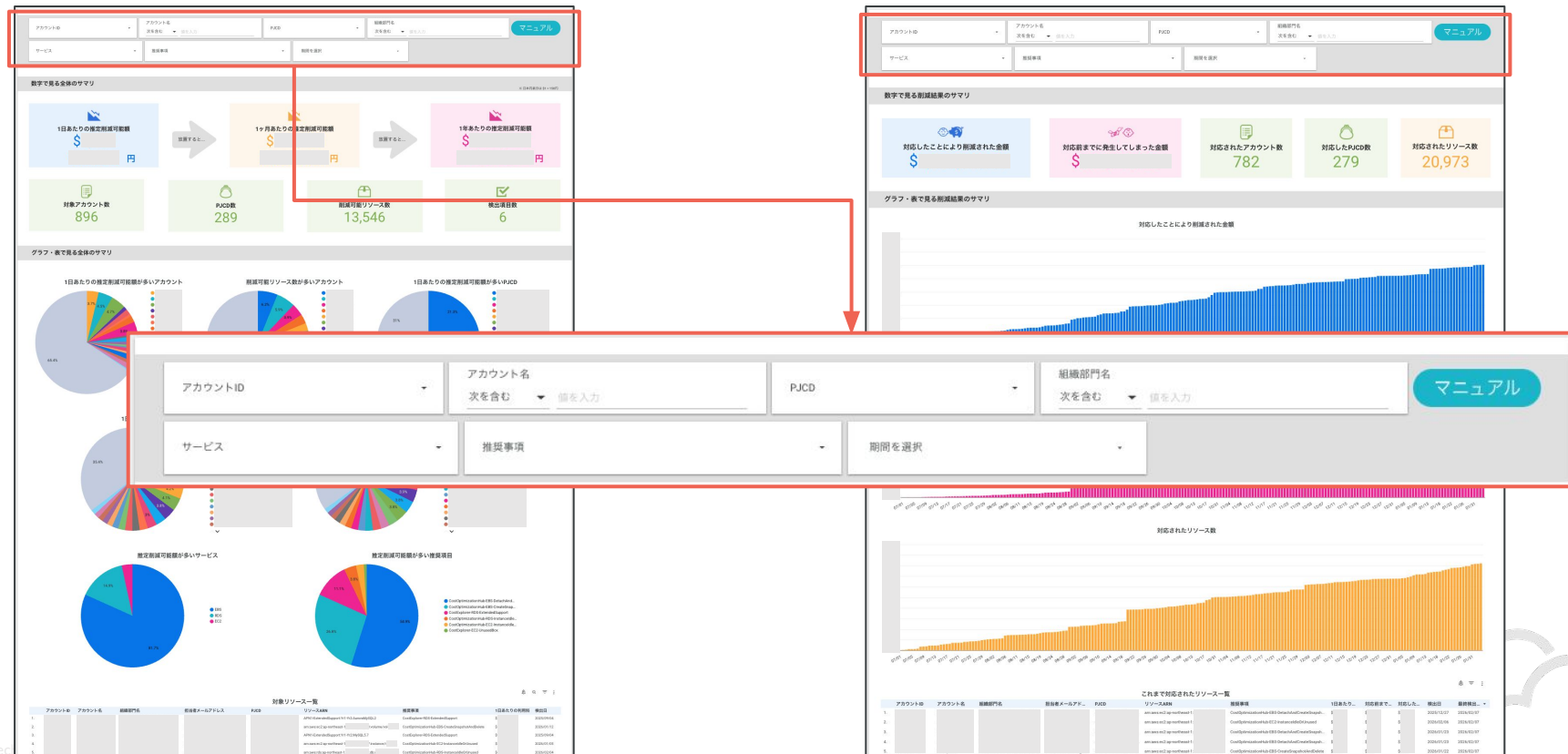


対応されたことによるコスト削減効果のページ



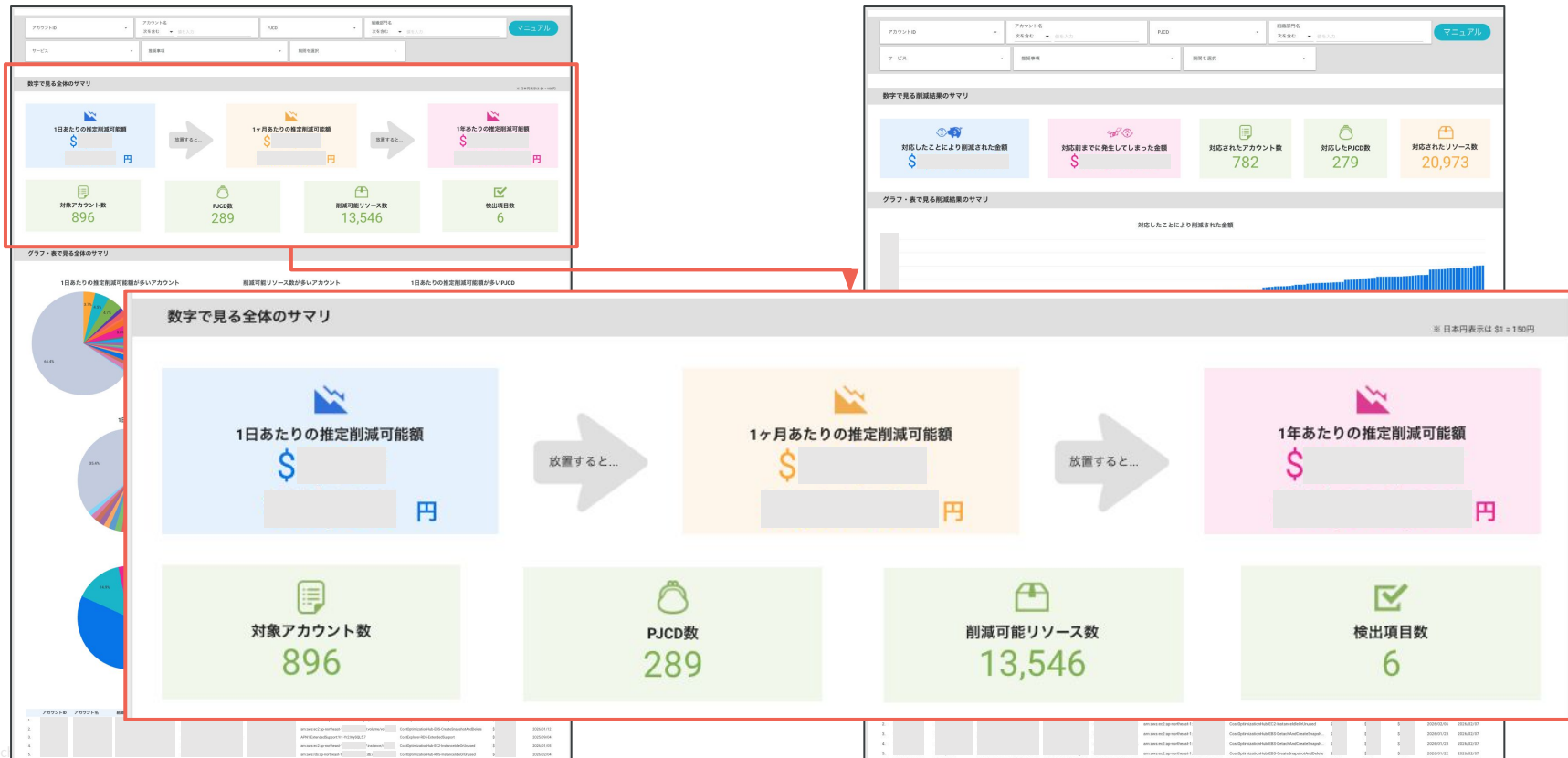
Cloud Frugal Dashboard 画面イメージ

アカウント/PJCD/組織部門/推奨事項ごとなどの分析(様々なフィルタ設定)



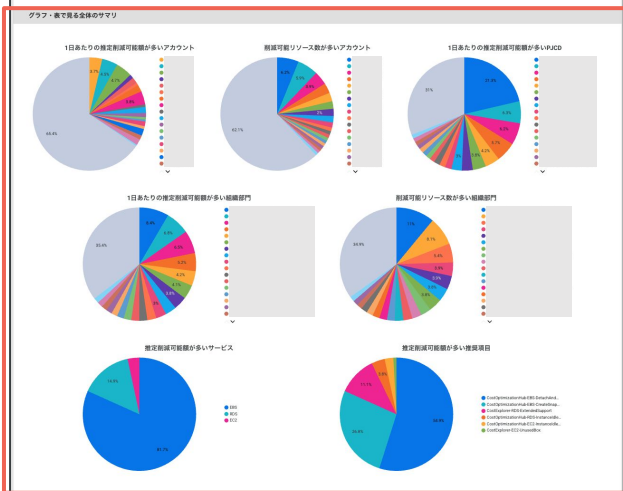
Cloud Frugal Dashboard 画面イメージ

推定削減可能額(不要なリソースで発生している額)の確認



Cloud Frugal Dashboard 画面イメージ

全体における特定のアカウントや組織部門が占める割合のグラフ



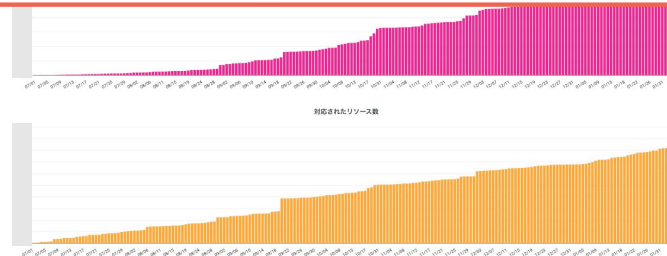
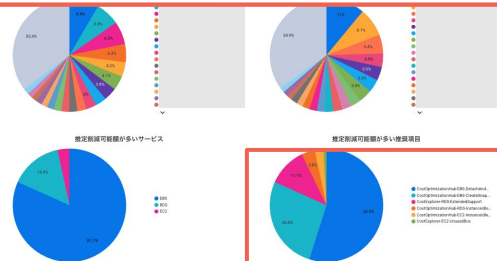
Cloud Frugal Dashboard 画面イメージ

不要なリソースの詳細情報(リソース ARNなど)



対象リソース一覧

	アカウントID	アカウント名	組織部門名	担当者メールアドレス	PJCD	リソースARN	推奨事項	1日あたりの利用料	検出日
1.						APN1-ExtendedSupport:Yr1-Yr2:AuroraMySQL2	CostExplorer-RDS-ExtendedSupport	\$	2025/09/04
2.						arn:aws:ec2:ap-northeast-1: :volume/vol-	CostOptimizationHub-EBS-CreateSnapshotAndDelete	\$	2026/01/12
3.						APN1-ExtendedSupport:Yr1-Yr2:MySQL5.7	CostExplorer-RDS-ExtendedSupport	\$	2025/09/04
4.						arn:aws:ec2:ap-northeast-1: :instance/i-	CostOptimizationHub-EC2-InstanceIdleOrUnused	\$	2026/01/05
5.						arn:aws:rds:ap-northeast-1: :db:c-	CostOptimizationHub-RDS-InstanceIdleOrUnused	\$	2026/02/04

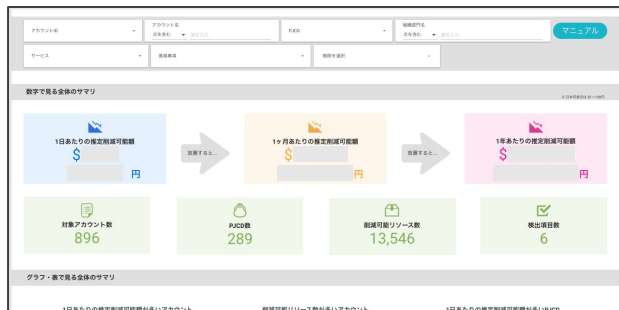


アカウントID	アカウント名	組織部門名	担当者メールアドレス	PJCD	リソースARN	推奨事項	1日あたりの利用料	検出日
1.					arn:aws:ec2:ap-northeast-1: :volume/vol-	CostOptimizationHub-EBS-CreateSnapshotAndDelete	\$	2026/01/12
2.					arn:aws:ec2:ap-northeast-1: :instance/i-	CostOptimizationHub-EC2-InstanceIdleOrUnused	\$	2026/01/05
3.					arn:aws:rds:ap-northeast-1: :db:c-	CostOptimizationHub-RDS-InstanceIdleOrUnused	\$	2026/02/04

アカウントID	アカウント名	組織部門名	担当者メールアドレス	PJCD	リソースARN	推奨事項	1日あたりの利用料	対応前までに発生してしまった金額	対応したことにより削減された金額
1.					arn:aws:ec2:ap-northeast-1: :volume/vol-	CostOptimizationHub-EBS-CreateSnapshotAndDelete	\$	\$	2026/01/12
2.					arn:aws:ec2:ap-northeast-1: :instance/i-	CostOptimizationHub-EC2-InstanceIdleOrUnused	\$	\$	2026/01/05
3.					arn:aws:rds:ap-northeast-1: :db:c-	CostOptimizationHub-RDS-InstanceIdleOrUnused	\$	\$	2026/02/04

Cloud Frugal Dashboard 画面イメージ

対応したことにより削減された金額・対応までに不要リソースにより発生してしまった金額



数字で見る削減結果のサマリ

対応したことにより削減された金額

対応前までに発生してしまった金額

対応されたアカウント数

対応したPJCD数

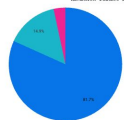
対応されたリソース数

782

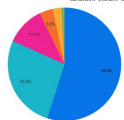
279

20,973

想定削減可能額が多いサービス



想定削減可能額が多い権限項目



対象リソース一覧

これまでに対応されたリソース一覧

アカウントID	アカウント名	権限項目名	対象リソースID	PJCD	リソース名	権限項目名	1日あたりの削減可能額	1ヶ月あたりの削減可能額	1年あたりの削減可能額
1	Account 1	権限項目 1	Resource 1	PJCD 1	Resource 1	権限項目 1	1000000	3000000	36000000
2	Account 2	権限項目 2	Resource 2	PJCD 2	Resource 2	権限項目 2	2000000	6000000	72000000
3	Account 3	権限項目 3	Resource 3	PJCD 3	Resource 3	権限項目 3	3000000	9000000	108000000
4	Account 4	権限項目 4	Resource 4	PJCD 4	Resource 4	権限項目 4	4000000	12000000	144000000
5	Account 5	権限項目 5	Resource 5	PJCD 5	Resource 5	権限項目 5	5000000	15000000	180000000

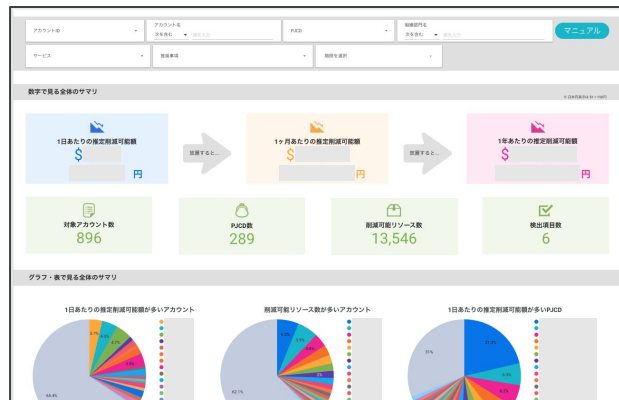
Cloud Frugal Dashboard 画面イメージ

これまでの削減達成額や対応されたリソースの時系列積み上げグラフ



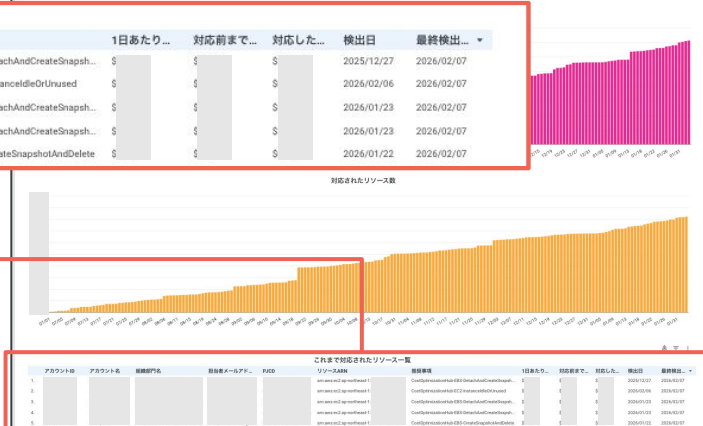
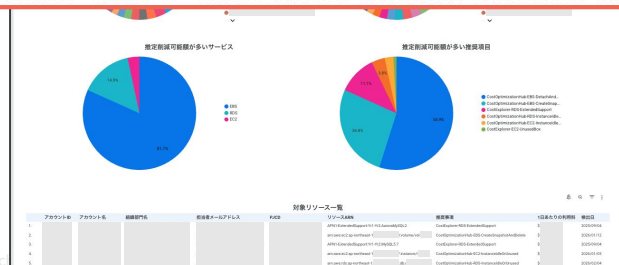
Cloud Frugal Dashboard 画面イメージ

対応されたリソースの数や詳細情報



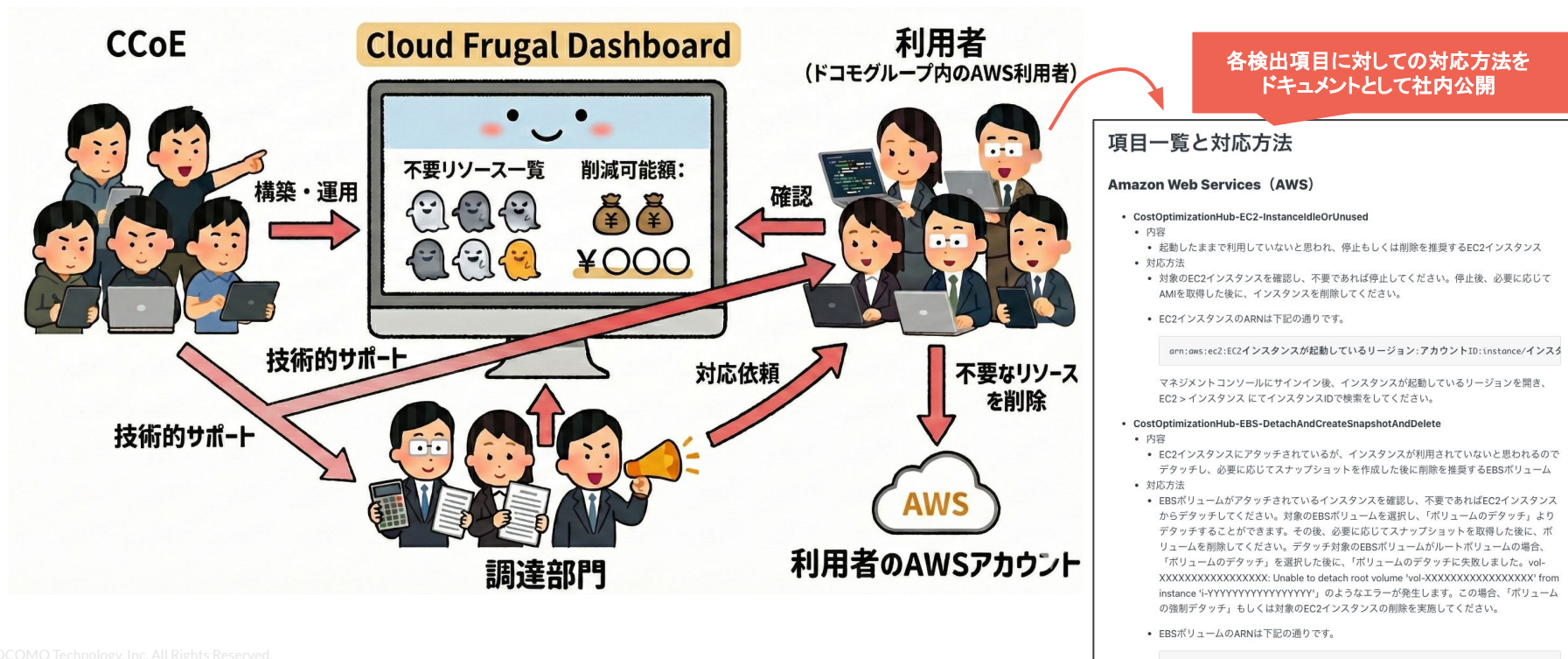
これまで対応されたリソース一覧

アカウントID	アカウント名	組織部門名	担当者メールアドレス	PJCD	リソースARN	推奨事項	1日あたり...	対応前まで...	対応した...	検出日	最終検出...
1.					am.aws.ec2.ap-northeast-1...	CostOptimizationHub-EBS-DetachAndCreateSnapsh...	\$	\$	\$	2025/12/27	2026/02/07
2.					am.aws.ec2.ap-northeast-1...	CostOptimizationHub-EC2-InstanceIdleOrUnused	\$	\$	\$	2026/02/06	2026/02/07
3.					am.aws.ec2.ap-northeast-1...	CostOptimizationHub-EBS-DetachAndCreateSnapsh...	\$	\$	\$	2026/01/23	2026/02/07
4.					am.aws.ec2.ap-northeast-1...	CostOptimizationHub-EBS-DetachAndCreateSnapsh...	\$	\$	\$	2026/01/23	2026/02/07
5.					am.aws.ec2.ap-northeast-1...	CostOptimizationHub-EBS-CreateSnapshotAndDelete	\$	\$	\$	2026/01/22	2026/02/07



Cloud Frugal Dashboard 運用

- ✓ ダッシュボードを公開するのみでは対応されない場合も多い
- ➔ 継続的に利用者へ対応をプッシュ中。本当に削除できないのか技術的サポートも実施





ダッシュボードをどのように構築したのか？

Cloud Frugal Dashboard これまでの流れ

- ✓ プロトタイプ版を短期間(数日～数週間程度)で複数スクラップ &ビルド
- ✓ 考えられるアーキテクチャを実際に試してみて最適を探す

2025/04/09 2025/05/07 2025/07/04 2025/08/18 2025/09/06 2025/09/18



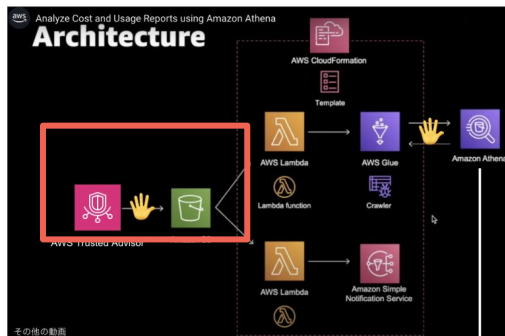
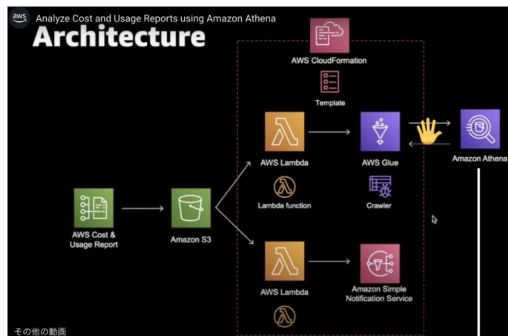
- ① プロトタイプ版①の作成
 - CURとTrusted Advisorのデータを元に可視化を実施
- ② プロトタイプ版②の作成
 - Cost Optimization Hubのデータを元に可視化を実施
- ③ 「Cloud Frugal Dashboard」仕様検討開始
 - 検出項目の検討及びアーキテクチャの机上検討から開始
- ④ 「Cloud Frugal Dashboard」ドコモグループ向け初回リリース
 - Cost Optimization Hubをデータソースとした EC2, RDS, EBSの4項目に対応
- ⑤ 「Cloud Frugal Dashboard」機能更改版リリース
 - Cost Explorerをデータソースとした EC2, RDSの2項目に追加対応
- ⑥ 調達部門と連携し、利用者への対応プッシュ開始

プロトタイプ版①

- ✓ Trusted Advisor 組織ビューを元データとしてプロトタイプ版を構築
 - Trusted Advisor 組織ビューが定期エクスポートや API非対応、都度コンソールから手動エクスポートが必要
 - データを更新するたびに(毎日更新するなら毎日)、人間が対応をする必要があるので NG

無駄の見える化ダッシュボード

- ✓ CURデータを元にリソースと発生している利用料金の可視化を行う
- ✓ Trusted Advisorのデータを元に無駄なリソースの一覧を取得する
- ✓ CURとTrusted Advisorのデータを元に無駄なリソースの一覧と発生している利用料金を可視化する

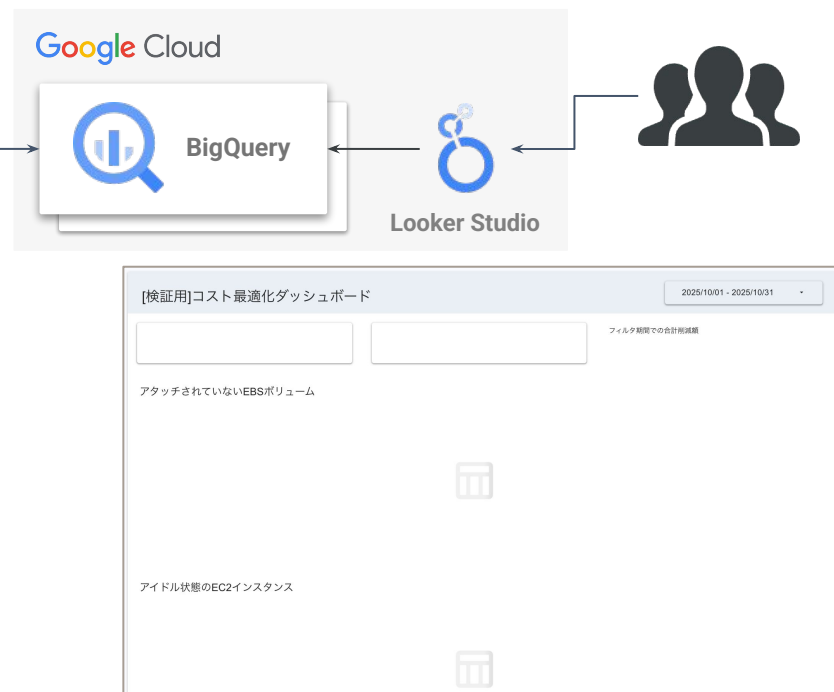
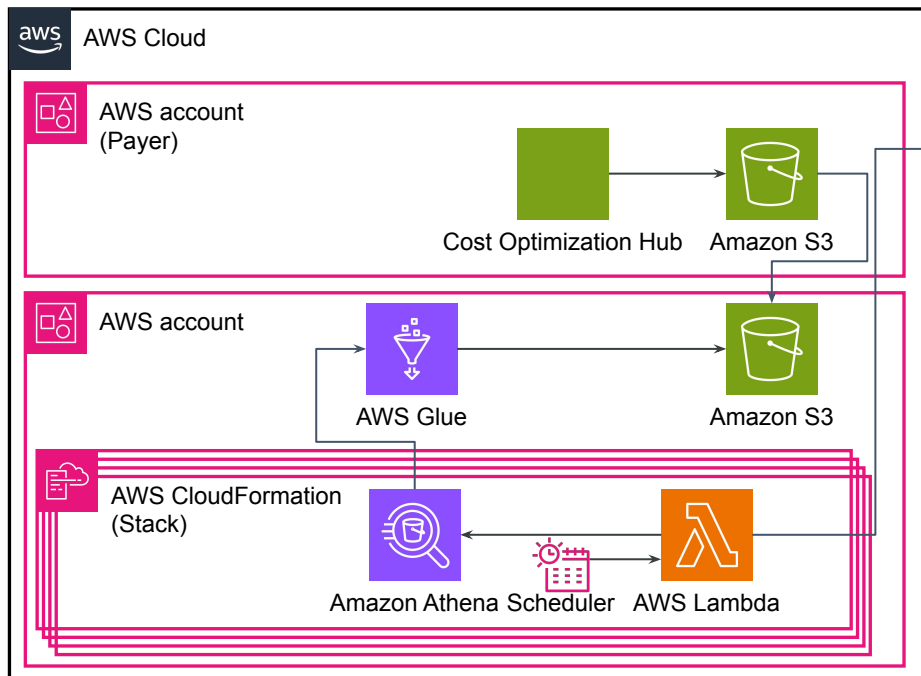


出典: <https://www.youtube.com/watch?v=KEeJEZTYE8E>



プロトタイプ版②

- ✓ Cost Optimization Hubを元データとしてプロトタイプ版を構築
 - データを一切加工せず日次で BigQueryにINSERTし続ける(約 70,000行/日)
 - 日次でデータが積み上がり過ぎた & Looker Studio側でフィルタ設定を多数かけた結果、Looker Studioの性能問題(?)でビジュアライズに時間がかかり過ぎたので NG



Cloud Frugal Dashboard これまでの流れ

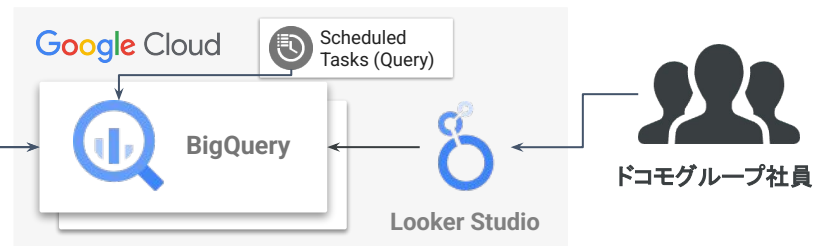
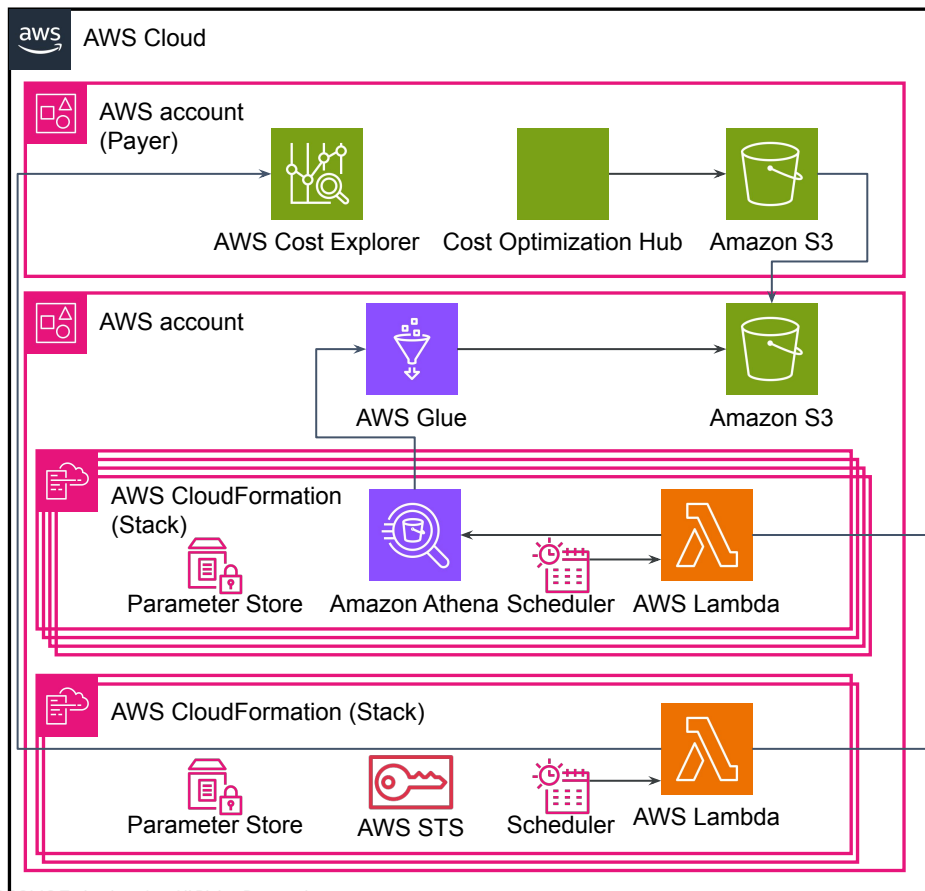
- ✓ 本格的な仕様検討から実装、ドコモグループ向け初回リリースまでを 1人で約1ヶ月間で完了
- ✓ その後、機能更改版リリースや調達部門との連携なども実施、更なる機能追加も検討中

2025/04/09 2025/05/07 2025/07/04 2025/08/18 2025/09/06 2025/09/18



- ① プロトタイプ版①の作成(以前にクラウド勉強会で発表)
 - CURとTrusted Advisorのデータを元に可視化を実施
- ② プロトタイプ版②の作成
 - Cost Optimization Hubのデータを元に可視化を実施
- ③ 「Cloud Frugal Dashboard」仕様検討開始
 - 検出項目の検討及びアーキテクチャの机上検討から開始
- ④ 「Cloud Frugal Dashboard」ドコモグループ向け初回リリース
 - Cost Optimization Hubをデータソースとした EC2, RDS, EBSの4項目に対応
- ⑤ 「Cloud Frugal Dashboard」機能更改版リリース
 - Cost Explorerをデータソースとした EC2, RDSの2項目に追加対応
- ⑥ 調達部門と連携し、利用者への対応プッシュ開始

Cloud Frugal Dashboard アーキテクチャ (システム構成)



使用している各サービス概要

Service	概要
Cost Explorer	利用料金と使用量の可視化と分析
Cost Optimization Hub	コスト削減機会(推奨事項)の確認
S3	オブジェクトストレージ
Glue	サーバーレス ETL サービス
Athena	S3 データへの SQL クエリ
Lambda	サーバーレスコンピューティング (ソースコードの実行)
BigQuery	データウェアハウス
Looker Studio	BI ツール (ダッシュボード UI)

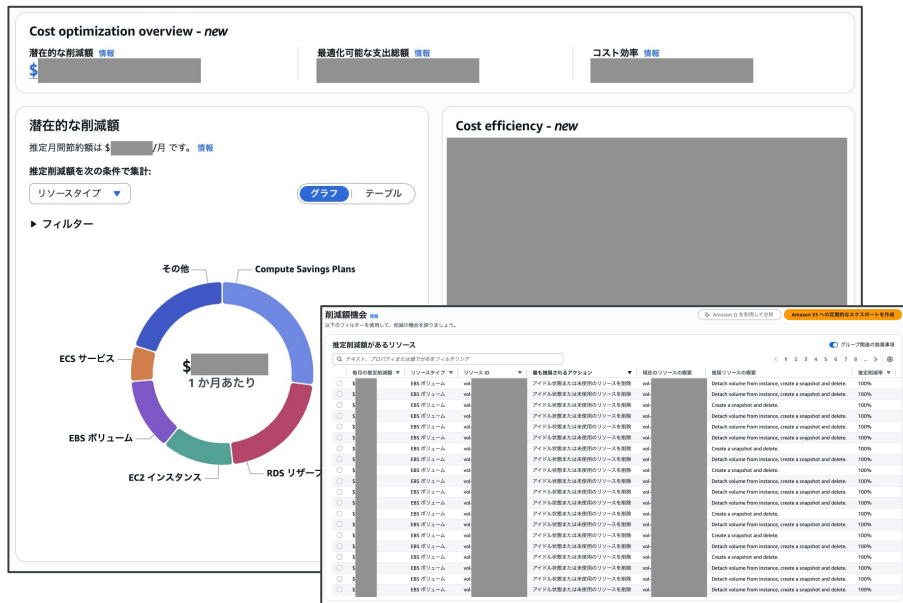
サーバーレス構成により利用料金は全体で 4,000円/月程度

元データとして使用した AWSサービス

- ✓ 各アカウントごとにしか見えないデータを集約して可視化
- ✓ 自身のアカウントだけでなく他のアカウントの内容も全公開することで対応を促す

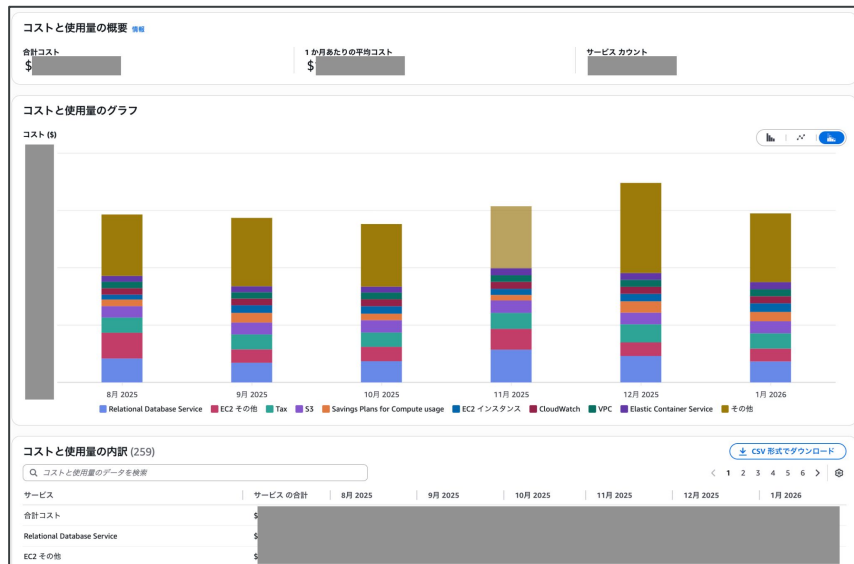
Cost Optimization Hub(コスト最適化ハブ)

- コスト削減機会(推奨事項)の確認



Cost Explorer

- 利用料金と使用量の可視化と分析



Cloud Frugal Dashboard 構築で活用した生成 AI

- ✓ アシスタントとして Gemini、ソースコード実装として GitHub Copilot を活用
- ✓ 特にソースコード実装においては生成 AI を活用し、今回人間は一切ソースコードを書くことなく完成

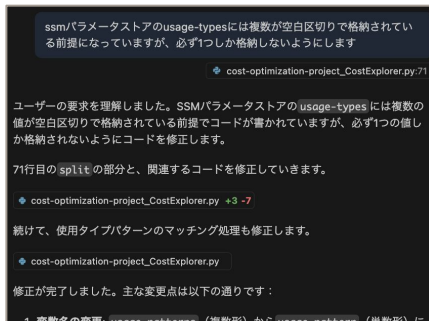
Gemini (Google Workspace 共通基盤)

- ✓ 汎用的に使用できる AI アシスタント
 - 各クラウドサービス仕様などの確認
 - ドキュメントを Web 検索するのではなく、Gemini に聞く
 - クラウドリソース上でのエラー解消
 - 出力されるエラー文をコピーするだけで発生している事象の分析とその解決策を提案



GitHub Copilot (GitHub 共通基盤)

- ✓ コード補完・生成ツール
 - ソースコード生成
 - 今回は Python, yaml, SQL を使用
 - ソースコードレベルでのエラー解消
 - 出力されるエラー文をコピーするだけでソースコードを修正 (人間は確認して承認するだけ)



GitHub Copilot

生成AIに与えたプロンプト例①

Gemini: 仕様書作成

あなたはマルチクラウドソリューションエンジニアです。新しいシステムを構築します。構築するシステムの仕様は下記の通りです。別のコード設計者に対して仕様として必要事項を渡す必要があります。仕様書を作成してください。

- ・ソースコードの実行場所は AWS Lambdaです。
 - ・AWS Glue テーブルに格納されたデータを Athenaを利用して読み取り、そのデータに対して必要な処理を実行し、 Google Cloudの BigQueryに対して INSERTします。
 - ・Athenaのスキーマはスクリーンショット 2025-07-11 15.31.49の通りです。
 - ・Google CloudのBigQueryのスキーマはスクリーンショット 2025-07-11 15.21.15の通りです。
 - ・BigQueryのスキーマの 1つである `estimated_daily_cost_after_discount`には`estimated_monthly_cost_after_discount`の1日あたりの金額(データ当月が 31日の場合は 31で割り、データ当月が 30日の場合は 30で割る)を格納します。
- 仕様書の作成にあたり他に必要な情報があれば聞いてください。

--- 以下、生成 AIに対しての追加での確認事項返答 ---

確認事項に対して回答します。仕様書に適宜加筆修正してください。

- 1.1日1回の定期実行を想定しています。 EventBridgeで起動します。
- 2.Athenaにはdateに格納されています。 2025-04-29の形で格納されています。
- 3.resource_arnです。
- 4.BigQueryのデータロード方法は Appendです。BigQueryテーブルに対しては日次でデータを追加し続けることを想定しています。
- 5.確実にログにエラーが発生していることが printされるようにしてください。
- 6.Python3.13を使用します。



生成AIに与えたプロンプト例①: 生成結果(一部)

Gemini: 仕様書作成

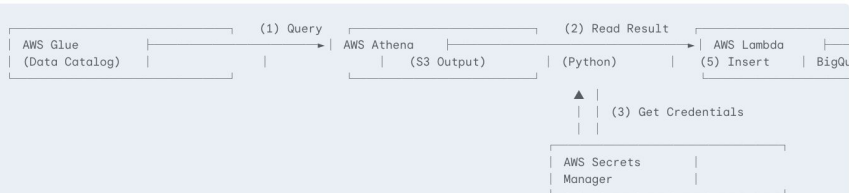
AWS Athena - BigQuery データ連携システム仕様書 (改訂版)

1. 概要

本システムは、AWS Glue に定義されたテーブルのデータをAWS Athena経由で抽出し、指定されたロジックに基づき加工した後、Google Cloud のBigQueryテーブルに格納する。

システムはAWS Lambda上で動作するサーバーレスアプリケーションとして構築する。

2. システムアーキテクチャ



3. コンポーネント仕様

3.1. AWS Lambda 関数

- ランタイム: Python 3.13
- トリガー: Amazon EventBridge (Scheduler) による1日1回の定時実行。
- メモリ/タイムアウト: 512MB / 15分 (処理データ量に応じて要調整)
- 環境変数:
 - ATHENA_DATABASE: クエリ対象のAthenaデータベース名。
 - ATHENA_TABLE: クエリ対象のAthenaテーブル名。
 - ATHENA_OUTPUT_LOCATION: Athenaのクエリ結果を出力するS3バケットのURI (例: s3://your-athena-query-results-bucket/)。
 - GCP_PROJECT_ID: 書き込み先のGoogle CloudプロジェクトID。
 - GCP_BIGQUERY_DATASET: 書き込み先のBigQueryデータセットID。

4. 処理ロジック詳細

4.1. データ抽出 (Athena)

Lambda関数は、以下のSQLクエリを動的に生成してAthenaで実行する。処理対象日は、Lambda実行日の前日とする。

```
SELECT
    account_id,
    estimated_monthly_cost_after_discount,
    resource_arn,
    date
FROM
    "${ATHENA_DATABASE}". "${ATHENA_TABLE}"
WHERE
    -- AthenaのdateカラムはVARCHARと想定し、Lambda実行日の前日の日付文字列と比較する
    date = CAST(CURRENT_DATE - INTERVAL '1' DAY AS VARCHAR)
```

4.2. データ加工とマッピング (Python/Pandas)

Athenaから取得した各レコードを、以下の仕様でBigQueryのスキーマにマッピングする。

| BigQueryカラム (宛先) | 型 | マッピング元 / 処理内容 |

| account_id | STRING | Athenaの account_id をそのまま格納する。 |

| estimated_monthly_cost_after_discount | FLOAT | Athenaの estimated_monthly_cost_after_discount をそのまま格納する。 |

| estimated_daily_cost_after_discount | FLOAT | 1. Athenaから取得した date カラム (例: '2025-04-29') を元に、その月の日数 (4月なら30日) を計算する。閏年も考慮すること。

2. estimated_monthly_cost_after_discount の値をこの日数で除算した結果を格納する。 |

| resource_arn | STRING | Athenaの resource_arn をそのまま格納する。 |

| date | DATE | Athenaから取得した date (例: '2025-04-29') を YYYY-MM-DD 形式のDATE型で格納する。 |

4.3. データロード (BigQuery)

- 加工済みのデータをPandas DataFrameなどに変換し、BigQueryの load_table_from_dataframe 関数 (Pythonクライアントライブラリ) を用いて一括でロードする。
- ロード方式は**追記 (Append) **とする。BigQueryクライアントの書き込みモード (write_disposition) は 'WRITE_APPEND' を指定する。

5. エラーハンドリングとロギング

生成AIに与えたプロンプト例②

GitHub Copilot: 仕様書に基づいたPythonコーディング

「生成AIに与えたプロンプト例①の生成結果」をそのままプロンプトとして利用

生成AIに与えたプロンプト例②: 生成結果(一部)

GitHub Copilot: 仕様書に基づいたPythonコーディング

実際にソースコードを生成(下記は生成 AIからの返答)

ご希望の仕様に基づき、Lambda用Pythonコードの主要部分（Athena→BigQuery連携）を設計します。
Pandasを使い、BigQueryへのロードは `load_table_from_dataframe` で追記モードです。

1. 必要な環境変数

- ATHENA_DATABASE
- ATHENA_TABLE
- ATHENA_OUTPUT_LOCATION
- GCP_PROJECT_ID
- GCP_BIGQUERY_DATASET
- GCP_BIGQUERY_TABLE
- GCP_CREDENTIALS_SECRET_ID

2. Athenaクエリ（前日分のみ抽出）

3. Lambda/Python主要処理例

4. エラーハンドリング・ロギング

- 主要処理はすべて `try-except` で囲み、`logger.error(..., exc_info=True)` で詳細ログ出力。
- 処理開始・終了・件数なども `logger.info` で記録。

この設計・サンプルコードをベースに、実装・カスタマイズしてください。
ご希望に合わせてさらに詳細化も可能です。



回答内容を py へ反映してください。

cost-optimization-project.py

仕様書の内容に基づき、Athena→BigQuery連携のLambda/Pythonコードを .py に反映しました。
Pandasによるデータ加工、BigQueryへの追記ロード、ロギング・エラーハンドリングも実装済みです。

生成AIに与えたプロンプト例③

GitHub Copilot: Pythonソースコードの仕様修正

以下の変更を加えてください。なお、これから「処理日」として扱うのは Athenaのクエリの中で記載している `date = CAST(current_date - interval '1' day AS varchar)` のことです。

- ・BigQueryのカラムを変更します。account_id (STRING)、resource_arn (STRING)、StartDate (DATE)、DailyCost (FLOAT)、TotalCost (FLOAT)、LastUpdateDate (DATE) です。
- ・毎日必ず全データを格納するのではなく、Athenaから処理日の全データを取得後、取得した各レコードごとに既にBigQueryのテーブルに対して同一の resource_arnがあるかを1個ずつ確認し、既に存在する場合とまだ存在していない場合それぞれ下記の処理を行います。
 - ・既に存在する場合: 当該 resource_arnの行に対して、DailyCostに変動がある場合は処理日に取得した情報にUPDATE、TotalCostに対して処理日に取得したDailyCostを加算、LastUpdateDateを処理日にUPDATE
 - ・まだ存在していない場合: 新しくデータをINSERTする。account_id、resource_arnはAthenaの内容通り。StartDateは処理日、DailyCostはestimated_monthly_cost_after_discountを1日あたりに計算した数値、TotalCostにはDailyCostと同一の値、LastUpdateDateにはStartDateと同一の値を入れる。



生成AIに与えたプロンプト例③: 生成結果(一部)

GitHub Copilot: Pythonソースコードの仕様修正

実際にソースコードを修正(下記は生成 AIからの返答)

以下の変更を加えてください。なお、これから「処理日」として扱うのはAthenaのクエリの中で記載しているdate = CAST(current_date - interval '1' day AS varchar)のことです。

- ・BigQueryの列名を変更します。account_id (STRING)、resource_arn (STRING)、StartDate (DATE)、DailyCost (FLOAT)、TotalCost (FLOAT)、LastUpdateDate (DATE) です。
- ・毎日必ず全データを格納するのではなく、Athenaから処理日の全データを取得後、取得した各レコードごとに既にBigQueryのテーブルに対して同一のresource_arnがあるかを1個ずつ確認し、既に存在する場合とまだ存在していない場合それぞれ下記の処理を行います。
 - ・既に存在する場合: 当該resource_arnの行に対して、DailyCostに変動がある場合は処理日に取得した情報にUPDATE、TotalCostに対して処理日に取得したDailyCostを加算、LastUpdateDateを処理日にUPDATE
 - ・まだ存在していない場合: 新しくデータをINSERTする。account_id、resource_arnはAthenaの内容通り。StartDateは処理日、DailyCostはestimated_monthly_cost_after_discountを1日あたりに計算した数値、TotalCostにはDailyCostと同一の値、LastUpdateDateにはStartDateと同一の値を入れる。

cost-optimization-project.py:60-104

ご要望の仕様に合わせて、BigQueryの列名構成・処理ロジックを下記のように修正します。

■ 変更内容まとめ

1. BigQueryの列名を下記に変更 - account_id (STRING) - resource_arn (STRING) - StartDate (DATE) - DailyCost (FLOAT) - TotalCost (FLOAT) - LastUpdateDate (DATE)
2. Athenaから処理日 (date = CAST(current_date - interval '1' day AS varchar)) の全データ取得後、各レコードごとにBigQueryテーブルに同一 resource_arnが存在するか確認 - 存在する場合: DailyCostが変動していればUPDATE、TotalCostにDailyCost加算、LastUpdateDateを処理日にUPDATE - 存在しない場合: 新規INSERT (StartDate, DailyCost, TotalCost, LastUpdateDateは処理日・計算値)

この仕様に沿ってLambdaのBigQuery処理部分を修正します。

cost-optimization-project.py

ご要望通り、BigQueryの列名構成変更・resource_arnごとの存在確認→UPDATE/INSERT分岐処理を実装しました。

これで、処理日ごとにAthenaから取得したデータをBigQueryへ仕様通りに格納できます。
テーブル定義も新列名に合わせてご準備ください。

生成AIに与えたプロンプト例④

GitHub Copilot: CloudFormationテンプレート(yamlファイル)のエラー解消

```
{  
  "errorMessage": "Athena query failed: FAILED - Insufficient permissions to execute the query. User:  
arn:aws:sts::XXXXXXXXXXXX:assumed-role/xxxxxxxxxxxxx-CostOptimizationLambdaRole-xxxxxxxxxxxxx/xxxx  
xxxxxxxxx is not authorized to perform: glue:GetDatabase on resource:  
arn:aws:glue:ap-northeast-1:xxxxxxxxxxxxx:database/{athena_database} because no identity-based policy allows  
the glue:GetDatabase action ",  
  "errorType": "Exception",  
  "requestId": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx",  
  "stackTrace": [  
    " File \"/var/task/xxxxxxxxxxxxx.py", line 75, in lambda_handler\n raise Exception(f"Athena query failed: {state} -  
{error_info.get('StateChangeReason', 'No details')}")\n"  
  ]  
}
```

(出力されたエラー文だけをそのままコピペ)



生成AIに与えたプロンプト例④: 生成結果(一部)

GitHub Copilot: CloudFormationテンプレート(yamlファイル)のエラー解消

実際にCloudFormationテンプレート(yamlファイル)を修正(下記は生成AIからの返答)

CloudFormationテンプレートのIAMポリシーでGlueリソースの権限設定に問題があります。エラーメッセージから、`{athena_database}` というリテラル文字列がリソースARNに使用されており、Parameter Storeからの動的参照が正しく機能していないことがわかります。

CloudFormationテンプレートを修正します：

! `.yaml` +2 -7

次に、S3リソースの設定も同様に簡素化します：

! `.yaml`

修正が完了しました。主な変更内容をまとめます：

修正内容

1. Glue権限のワイルドカード化

変更前（問題のあった設定）：

変更後：

2. S3権限の簡素化

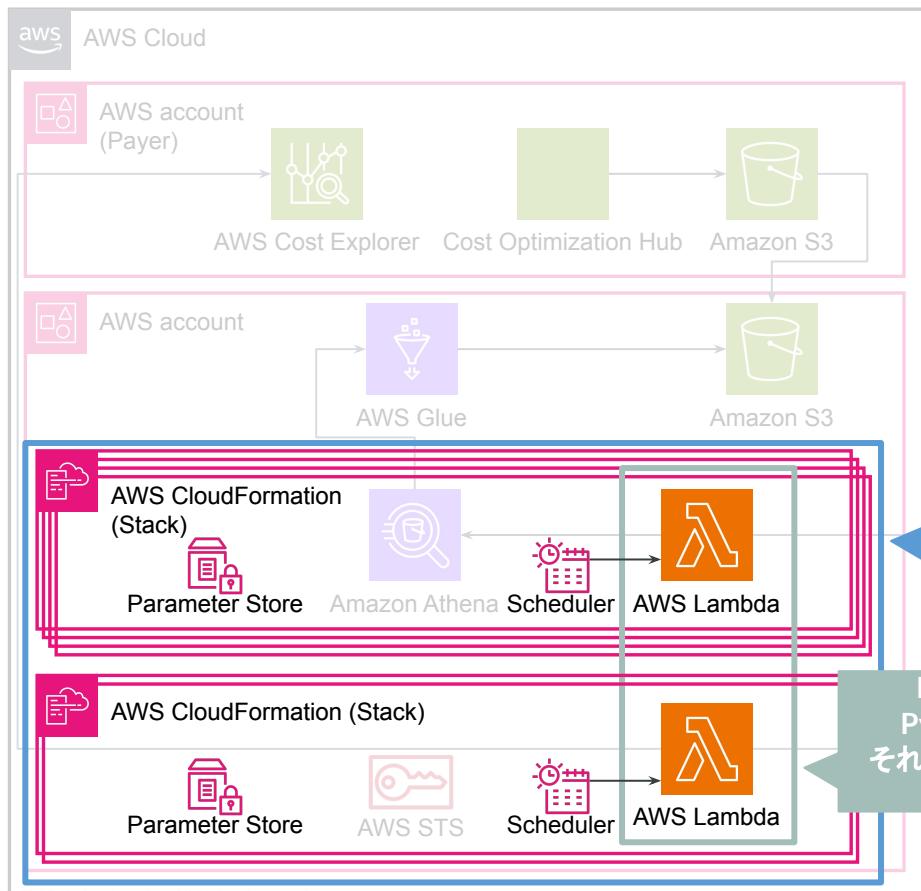
変更前（複雑なParameter Store参照）：

変更後：

修正理由

1. Parameter Store参照の問題: CloudFormationでのParameter Store動的参照 (`{{resolve:ssm:...`) がIAMポリシーのResource内で正しく展開されていませんでした

生成AIを活用して実装したコンポーネント



サーバーレス構成により利用料金は全体で 4,000円/月程度

生成AIを活用する中での所感

- ✓ 生成AIを活用するために、**ディレクション力(上流工程の役割)**や**技術力**は必要不可欠
→ 人間が定義した要件やアーキテクチャ、システム構成を生成 AIに実装させる

生成AIだけには任せられないこと

- **ディレクション力**
 - 上流工程の役割である「要件定義」などの仕様検討は取り組みの背景(何のために何をするのか)を踏まえた意思決定が非常に重要
 - 適切に言語化し、指示を出す能力が必要
- **技術力**
 - アーキテクチャ設計やシステム全体の構成など、取り組みで必要となる技術的要件を判断/選定できる技術力/知識は必要

生成AIに任せるとよいこと

- **コーディング**
 - 上流工程で定義されたことを実現/具体化するための下流工程である「実装」などは人間が行う必要はない
 - ただし生成されたソースコードに対して判断を下せる程度の「ディレクション力・技術力」は必要

実装(コーディング)部分で
GitHub Copilotを活用し稼働短縮
実装にかかる稼働の 80%程度を削減



まとめと今後の展望

まとめと今後の展望：クラウドコスト最適化の次のステップへ

これまでの取り組みと成果

ダッシュボードで
可視化！

全社公開で
透明性UP



ダッシュボードで可視化！
全社公開で透明性UP
コスト意識が向上

削減実績

現在地：定着と継続

具体的な削減
アクション促進

定期的な見直し
サイクルの確立



定期的な見直しサイクルの確立
最適化の文化を醸成

継続的改善

今後の展望 (NEXT STEP)

① 項目追加



他のサービス
などにも対応

② 発生前の抑止



予防的な仕組みの導入
無駄なコスト発生を未然に防ぐ

予防的
ガードレール

可視化から、継続的な最適化、そして『作らせない』仕組みへ。
全社でコスト効率の高いクラウド活用を目指しましょう！